



Meet Python Turtle

Ontario Math C3.1 — repeating events, now in typed code

Name:

Date:

★ I can read a typed loop that makes Bit move again and again.

Same loop — now typed

You already used a repeat block to make Bit do something many times. Python Turtle does the SAME thing — but you TYPE it instead of dragging blocks.

Here is the repeat you know, and the same idea written in Python. Bit is the turtle that draws.

The block you know

repeat 3

move 20

Move 20, three times.

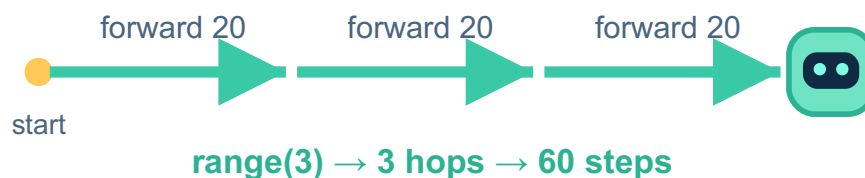
The same thing in Python

```
for step in range(3):  
    bit.forward(20)
```

PYTHON

OUTPUT

Bit moves forward 20, three times → 60 steps.



range(3) means do the inside 3 times — Bit makes 3 hops of 20.

1 Read it

Look at the Python code. `range(3)` repeats the inside 3 times. How many times does Bit move forward? ____ Each hop is 20, so how far in TOTAL? ____

2 Predict, then run

You change `range(3)` to `range(5)`. PREDICT: how many hops, and how far in total? Then run it to check.

3 Write your own

Make Bit move forward 10, FOUR times. Fill in the two numbers: for step in `range(____)`:
`bit.forward(____)`

4 Challenge

Bit's code is for step in `range(4)`: `bit.forward(25)`. How far does Bit travel in total? Show how you worked it out.

Big idea

`range(n)` in a Python loop is just like repeat n in blocks — it does the inside n times. Typed code can repeat, exactly like blocks can.



Meet Python Turtle

Quick facilitation notes & answer key

What this worksheet teaches

Students meet typed code for the first time. They read a short instruction that means "repeat 3 times: move forward 20" and work out how many hops Bit makes and how far it travels. It is the same repeat idea they used with blocks — now typed. No coding experience needed.

Before you start — showing the drawing (optional)

The worksheet shows the typed instructions AND a picture of how far Bit hops, so you can teach straight from the printout — no computer required.

To show it live (optional): open a browser, go to a free "Python turtle" playground (e.g. search "trinket python turtle"), type the few lines from the worksheet, and press Run — a turtle draws it. You only press Run.

How to lead it (about 30 minutes)

1. Connect to what they know. Remind them of the "repeat" block from block coding — the typed instruction does the same job.
2. Show it (you do). Read the instruction aloud: "repeat 3 times: move forward 20." Count the hops together — 3 hops of 20.
3. Let them work. Students do Exercises 1–4 (Ex 4 is the stretch), solo or in pairs.
4. Wrap up: the repeat number tells Bit how many times to do the line below it — typed code can repeat, just like the blocks.

Answer key (these are the verified correct answers)

Exercise 1 — 3 hops; 60 steps in total (3×20).

Exercise 2 — 5 hops; 100 steps in total; the prediction should be 100.

Exercise 3 — the missing numbers are 4 (repeat 4 times) and 10 (move forward 10).

Exercise 4 (challenge) — 100 steps in total (4×25).

If students get stuck — and ways to adjust

Watch for: reading "repeat 3" as 4 (counting an extra). It means exactly 3 times.

Make it easier: point at the indented line and say "do this line this many times."

Make it harder: ask the total steps for a "repeat 6, move forward 10" instruction (60).

Ontario curriculum link (official wording)

C3.1 — solve problems and create computational representations of mathematical situations by writing and executing code, including code that involves sequential, concurrent, and repeating events.

In plain terms: students write and run step-by-step instructions — including a "repeat" instruction — to make something happen.

C3.2 — read and alter existing code, including code that involves sequential, concurrent, and repeating events, and describe how changes to the code affect the outcomes.

In plain terms: students read instructions, change one, and explain what the change did to the result.

You'll know it worked when

The student reads the repeat instruction as "repeat n times", counts the hops and total distance (Ex 1–2), fills in the repeat line (Ex 3), and works out a total from a typed loop (Ex 4).